

Diffusion-Based Refinement for Robust Long-Horizon Predictions with Neural Operators

Bachelor Thesis

Submitted By

Noel Christoph Kompalla

Student of

BSc Mechanical Engineering

Department of Mechanical and Process Engineering

19. December 2025

Supervised By

Prof. Dr. Laura De Lorenzis

Lukas Brock

Abstract

Conventional numerical solvers for time-dependent partial differential equations are computationally expensive, motivating surrogate models for fast inference. Neural Operators such as the Fourier Neural Operator (FNO) and Convolutional Neural Operator (CNO) can efficiently predict spatio-temporal fields, but autoregressive long-horizon rollouts often degrade due to error accumulation.

In this thesis, we evaluate diffusion-based iterative refinement with a Neural Operator backbone for long-horizon temperature-field prediction in a laser-cutting setting. We benchmark against FNO and CNO baselines and study two variants, with and without input perturbation. On a 165 time-step rollout, input perturbation achieves average relative errors of 8.10% rel. L_1 , 13.01% rel. L_2 . Overall, while input perturbation improved stability, neither refinement variant achieved consistent performance gains over the baseline operators for this problem setting.

Table of Contents

List of Abbreviations

1	Introduction	1
2	Theoretical Background	2
2.1	Neural Operator	2
2.1.1	Formulation of Neural Operators	2
2.1.2	Fourier Neural Operator	3
2.1.3	Convolutional Neural Operator	3
2.2	Diffusion-Based Refiner	4
2.2.1	Training	5
2.2.2	Inference	6
2.3	Heat Equation	6
2.3.1	The Thermal Model	7
3	Methodology	8
3.1	Problem Setting	8
3.2	Dataset and Dataloader	8
3.2.1	Dataset	8
3.2.2	Dataloader	9
3.3	Diffusion-Based Refinement	9
3.3.1	v-prediction	10
3.4	Metrics	11
3.4.1	Relative L_p Norms	11
3.4.2	Mean Squared Error	11
3.4.3	Coefficient of Variation	12
3.4.4	Logarithmic-Transformed Mean Absolute Deviation	12
4	Experiments	13
4.1	Baseline	13
4.1.1	Baseline - Experimental Setup	13
4.1.2	Baseline - Models	13
4.1.3	Baseline - Results	14
4.1.4	Baseline - Training Dynamics	15
4.2	Diffusion-Based Refiner - Modification Open-System Conditioning	16

4.2.1	Modification Open-System Conditioning - Experimental Setup	17
4.2.2	Modification Open-System Conditioning - Model	17
4.2.3	Modification Open-System Conditioning - Results	19
4.2.4	Modification Open-System Conditioning - Training Dynamics	20
4.3	Diffusion-Based Refiner - Modification Training Stabilization	21
4.3.1	Modification Training Stabilization - Experimental Setup	21
4.3.2	Modification Training Stabilization - Model	22
4.3.3	Modification Training Stabilization - Results	22
4.3.4	Modification Training Stabilization - Training Dynamics	24
4.4	Training and Inference Speeds	25
4.5	Summary of Results	25
5	Conclusion	27
5.1	Key Findings	27
5.2	Limitations	28
5.3	Future Work	29
5.4	Overall Conclusion	29
6	Acknowledgment	30
A	Appendix	31
A.1	Modification Open-System Conditioning Pseudo Code	31
A.2	Modification Training Stabilization Pseudo Code	32
A.3	Inference Pseudo Code	33
A.4	Baseline Results	34
B	Hyperparameters	34
B.1	FNO Baseline Hyperparameter	34
B.2	CNO Baseline Hyperparameter	35
B.3	DBR Hyperparameter	35
	Bibliography	36

List of Abbreviations

Abbreviation	Description
CNO	Convolutional Neural Operator
CV	Coefficient of Variation
DBR	Diffusion-Based Refiner
DDPM	Denoising Diffusion Probabilistic Model
FFT	Fast Fourier Transform
FNO	Fourier Neural Operator
LR	Learning Rate
log-MAD	Logarithmic-Transformed Mean Absolute Deviation
MSE	Mean Squared Error
PDE	Partial Differential Equation

1 Introduction

Numerical simulations of time-dependent partial differential equations are essential in many engineering applications, but high-fidelity solvers can be computationally expensive, particularly when repeated evaluations are required for design iteration, optimization, or real-time monitoring. This motivates surrogate models that approximate the underlying solution operator to enable fast inference.

Neural Operators [1] have emerged as a promising class of surrogates because they aim to learn mappings between function spaces rather than between fixed-dimensional vectors, which makes them well suited for predicting spatial fields such as temperature distributions. However, in autoregressive long-horizon rollouts, small single-step errors can accumulate over time and may lead to unstable or inaccurate trajectories [2], especially in settings with non-stationary dynamics or changing forcing.

Recent work proposed diffusion-based refinement to improve long-horizon predictions by iteratively denoising and refining a model’s estimate instead of relying on a single deterministic forward step [2]. This thesis¹ investigates whether diffusion-based refinement can improve the robustness and long-horizon rollout accuracy of neural operator predictions of laser-cutting temperature fields, characterized by a localized moving heat source and steep thermal gradients. We benchmark established neural operator baselines and evaluate two variants of a diffusion-based refinement approach, including a perturbation-based modification intended to improve stability.

The remainder of this thesis is structured as follows: Chapter 2 introduces the theoretical background on neural operators, diffusion-based refinement, and the underlying heat equation. Chapter 3 describes the problem setting, dataset, model architectures, and evaluation metrics. Chapter 4 presents the experimental results, including baseline benchmarks, refinement attempts, and computational cost comparisons. Chapter 5 concludes with key findings, limitations, and directions for future work.

¹Code for the experiments and training pipeline: https://github.com/realnoel/BA_2D_Laser.

2 Theoretical Background

This chapter provides the theoretical background required to understand neural operators and introduces the two architectures used in this work. Furthermore, the underlying partial differential equation (PDE) is explained, and the procedure for generating the dataset is described.

2.1 Neural Operator

Neural Operators learn mappings between infinite-dimensional function spaces, such as PDE input fields and their corresponding solution fields, while representing these functions through finite-dimensional spatial grids [1], [3].

A key motivation for neural operators is their ability to approximate nonlinear operators in infinite-dimensional function spaces rather than finite-dimensional mappings. Universal approximation results for neural networks were established earlier and have since been extended to operator and functional approximation [4]. Building on this theoretical foundation, DeepONet [5] proposes a concrete operator-learning architecture and establishes a universal approximation result, showing that compact neural architectures can approximate a broad class of nonlinear operators from data.

Kovachki et al. [1] introduced a class of neural operators that are mesh-independent and discretization invariant. Instead of learning a map tied to a specific grid resolution, these architectures learn the underlying operator itself, enabling the model to generalize across discretizations and spatial resolutions.

2.1.1 Formulation of Neural Operators

Let $X = \{a : D \rightarrow \mathbb{R}\}$ and $Y = \{u : D' \rightarrow \mathbb{R}\}$ denote the spaces of input and output functions, where $D, D' \subset \mathbb{R}^d$ are bounded domains. The objective is to approximate a solution operator $\mathcal{G}^\dagger$ for

$$\mathcal{G} : X \rightarrow Y \tag{1}$$

which maps an input function (e.g., initial or boundary conditions, source terms, or PDE coefficients) to its corresponding solution function [1], [3].

2.1.2 Fourier Neural Operator

The Fourier Neural Operator (FNO) [3] is one of the first practical architectures within the neural operator framework introduced by Kovachki et al. [1]. It combines global convolutions in the Fourier domain with local linear transformations in physical space to learn the underlying operator.

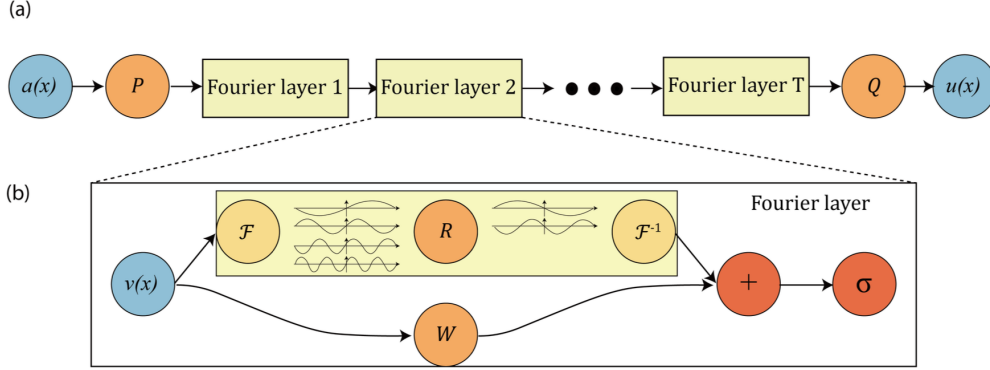


Figure 1: Schematic of the FNO architecture, reproduced from [3], including the lifting operator P to the feature space, the projection operator Q to the solution field, and the stacked Fourier layers operating in the spectral domain.

FNO approximates the solution operator $\mathcal{G}^\dagger$ through a sequence of Fourier layers. The model first lifts the input function $a \in X$ via a local transformation into a higher-dimensional feature space v . Each Fourier layer then applies a two-branch operation consisting of a kernel integral transformation in the Fourier domain and a local linear transformation in physical space. Concretely, the feature representation v is transformed using a Fast Fourier Transform (FFT), high-frequency modes are truncated, and the remaining spectral coefficients are linearly parameterized before being mapped back through an inverse FFT. In parallel, a pointwise linear transformation is applied directly to v . The outputs of the spectral and local branches are combined and passed through a nonlinear activation function σ . After the final Fourier layer, the resulting representation v is projected via a local transformation to the solution field $u \in Y$.

2.1.3 Convolutional Neural Operator

The Convolutional Neural Operator (CNO) [6] is a neural operator architecture inspired by the U-Net design and developed to overcome several limitations of previous convolution-based operator learning methods. In contrast to earlier approaches, the CNO is alias-free² [7], structure-

²Aliasing refers to a distortion that arises when a function is sampled on a discrete grid, causing unresolved high-frequency components to be incorrectly represented as lower frequencies.

preserving, and provides universal approximation guarantees for a broad class of partial differential equations. Its architecture consists of a hierarchy of convolutional layers that progressively capture lower-frequency components as the resolution decreases, enabling both stable training and efficient representation of multi-scale physical phenomena.

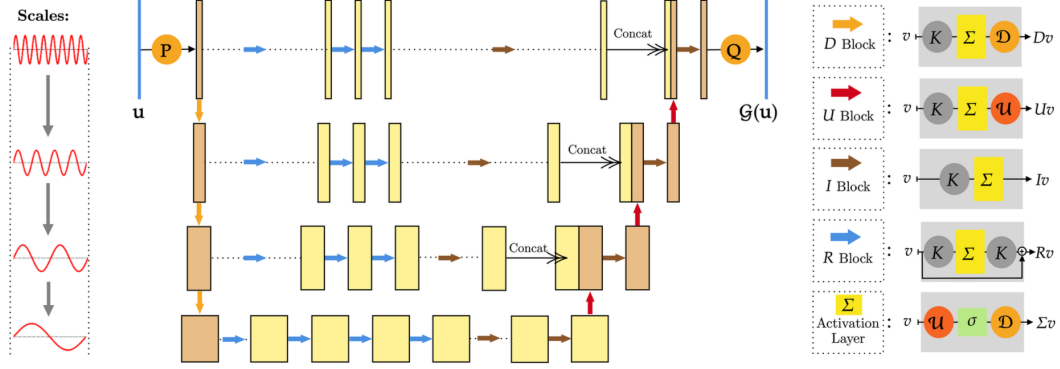


Figure 2: Schematic of the CNO architecture, reproduced from [6], including the lifting operator P to the feature space, the projection operator Q to the solution field, and the coupled down-/upsampling blocks with skip connections.

CNO approximates the solution operator $\mathcal{G}^\dagger$ through a multiscale architecture. First, the input function is lifted into a higher-dimensional feature space. The model then processes the lifted representation through a sequence of layers that progressively downsample the spatial resolution. Each downsampling stage applies a bandlimited convolution operator followed by a nonlinear activation, which extracts features at increasingly coarse scales. In each scale, additional features are extracted through several bandlimited kernel operations and nonlinearities that operate without altering the spatial resolution. These parallel paths help stabilize the operator approximation and retain information that would otherwise be lost during downsampling.

After reaching the coarsest scale, the architecture reverses this process through corresponding upsampling stages. Each upsampling layer increases the spatial resolution and combines the coarse features with the fine-scale information transmitted through skip-connections from the downsampling path. This restores high-frequency details while keeping the global operator structure learned at lower resolutions. Finally, a projection layer maps the multi-scale feature representation back to the function space, producing the operator output.

2.2 Diffusion-Based Refiner

The diffusion-based refiner (DBR) is based on the PDE-Refiner introduced by Lippe et al. [2]. Its framework includes an iterative refinement module, implemented either as an autoregressive

model or as a diffusion-based model. In this thesis, the diffusion-based approach is used. The authors argue that single-step models trained with mean squared error (MSE) tend to minimize error by primarily learning the low-frequency, high-amplitude dynamics of the system. However, in long-horizon predictions, high-frequency, low-amplitude dynamics can deviate substantially, and the resulting errors may accumulate over time. See Fig. 3.

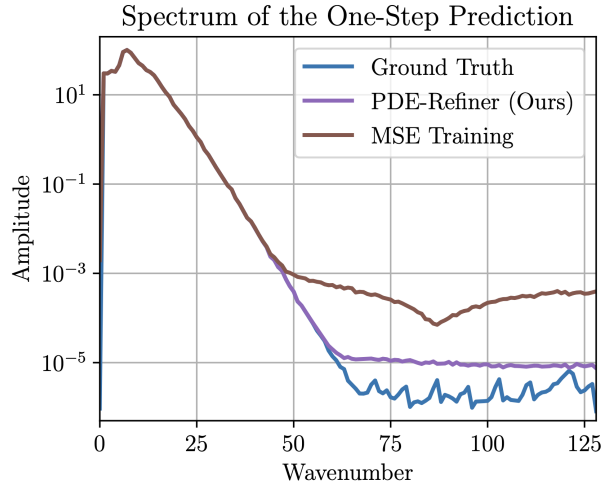


Figure 3: Spectral comparison of the PDE-Refiner and MSE-trained baseline against the ground truth for the Kuramoto-Sivashinsky equation, reproduced from [2].

2.2.1 Training

The model is trained to iteratively refine a residual estimate across refinement steps for the input x . Composing these updates over K refinement steps, yields a final residual r^K such that $y \approx x + r^K$, to approximate the target y .

This is done by computing the residual between the input and the target. The model can then either learn this residual explicitly or rely on an optimized scheduler to define the refinement targets at each step. The noise scheduler controls the strength of the added noise for the selected refinement step k and under the v-prediction parameterization [8], deterministically defines the corresponding velocity target. The model is trained to predict this step-dependent velocity, which mixes data and noise and can improve stability during training and inference, as described in Section 3.3.1. Finally, the loss for the training step is computed.

2.2.2 Inference

First, an initial residual estimate is initialized as random Gaussian noise. The model then performs an iterative refinement procedure in which, at each iteration, it predicts a residual update that is accumulated internally. After completing all refinement steps, the accumulated residual is added to the original input to obtain the refined estimate. This final estimate represents the model’s output and is used as the prediction for the next timestep.

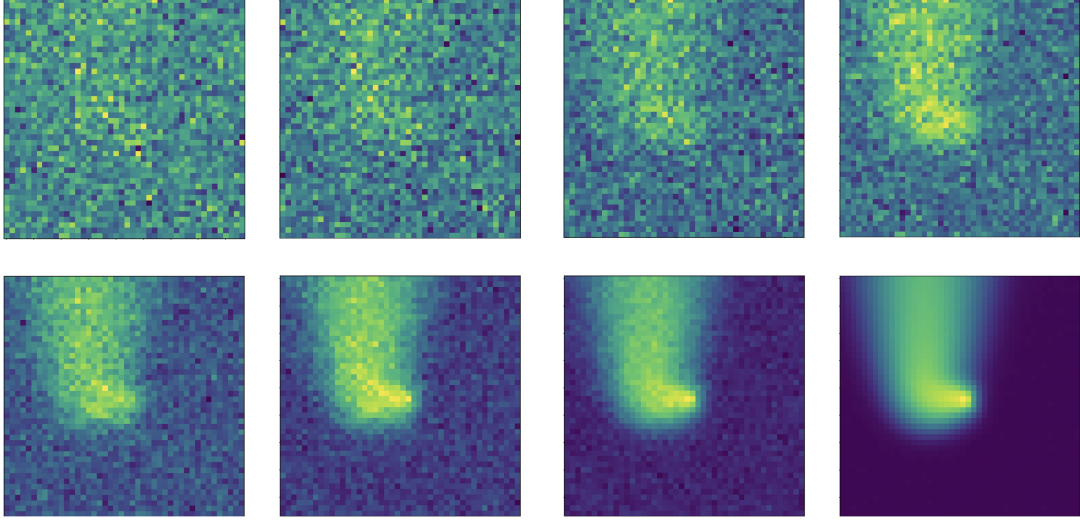


Figure 4: Illustration of the iterative denoising process, showing the intermediate reconstructions $x + r^k$ at each refinement step k , from an initial noisy residual estimate (top left) to the final output (bottom right).

2.3 Heat Equation

The governing equation is a linear parabolic PDE, second-order in space and first-order in time, which describes the evolution of the temperature field as shown in Eq. (2) [9].

$$\rho c \frac{\partial T}{\partial t} = \lambda \nabla^2 T + \dot{Q}''', \quad (2)$$

where ρ denotes the material density, c the specific heat capacity, ∇ the spatial gradient operator, λ the (temperature-dependent) thermal conductivity, and $\dot{Q}'''$ the volumetric heat source.

2.3.1 The Thermal Model

The thermal model used in this work follows the formulation of Moreira et al. [10]. In their approach, the energy balance is first expressed in its strong form using the enthalpy rate and Fourier's law. This strong form is then reformulated into the corresponding weak form, which is solved using the finite element method.

The model includes two primary heat-loss mechanisms: (i) conductive heat transfer at the interface between the substrate and the deposited material, and (ii) radiative and convective heat exchange with the surrounding environment.

3 Methodology

This chapter outlines the experimental methodology of this thesis. We first define the laser-cutting prediction task and summarize the dataset and preprocessing. Next, we describe the dataloader strategies and the considered model families (baseline neural operators and the diffusion-based refiner), including the key modifications introduced for the open-system setting. The chapter concludes by specifying the training procedure and the metrics used to quantify prediction accuracy and long-horizon rollout stability.

3.1 Problem Setting

The underlying problem is to predict the temperature distribution during laser cutting in a metal plate by learning the solution of the underlying PDE from given data. The setup considers an infinite metal plate initially at zero temperature. A laser moving with $v = 600$ mm/min, represented by a point heat source, introduces energy into the system, resulting in a time-dependent temperature distribution within the plate. For simplicity, we use a co-moving reference frame in which the laser is fixed at the domain center, and we restrict prediction to this frame.

Parameter	Description	Value
ρ	Density	7800 kg/m ³
c	Specific heat capacity	550 J/(kg · K)
λ	Thermal conductivity	20 W/(m · K)
q	Heat transfer coefficient	1 W/(m ² · K)
ϵ	Radiation emissivity	0.7

Table 2: Material parameters used throughout this work.

3.2 Dataset and Dataloader

3.2.1 Dataset

The dataset was obtained from a numerical solver, see [2.3.1](#), which computes the solution to the underlying PDE. It comprises eight trajectories, which are divided into a training set of seven trajectories and a test set containing one trajectory. In the training set, the number of time-step samples per trajectory ranges from 88 to 726, resulting in a total of 1,934 samples, while the test set contains 170 time-step samples.

Each sample contains four channels. Two representing the physical quantities, temperature and

laser power and two corresponding to the shift of the reference frame in x - and y -directions. The physical quantities are converted to a natural logarithmic scale, with temperature defined as $T = \ln(T)$ and power as $Q = \ln(Q + 1)$. Further the physical quantities are represented on a spatial grid of size 44×44 .



Figure 5: Testing trajectory with 165 time-steps. The arrow indicates the direction of laser motion.

3.2.2 Dataloader

This thesis uses a custom data loading pipeline that was originally derived from public PyTorch code examples and adapted to the needs of our dataset and the proposed dataloader strategies. It provides four distinct dataloader strategies, which differ in how they construct the forcing-term inputs and the prediction targets.

These strategies can be grouped into two categories: single-step prediction, where the model predicts only the next timestep $t + 1$, and multi-step prediction, where the model predicts a sequence $t + 1, \dots, t + N$. The dataloader parameter N determines the temporal extent of the returned sample. In single-step strategies, N controls only the number of past timeframes included as input, whereas in multi-step strategies it additionally defines the number of future target frames. Figure 6 illustrates the strategies in detail.

3.3 Diffusion-Based Refinement

The original PDE-Refiner formulation [2] is evaluated on approximately closed dynamical systems, where the next state is primarily determined by the previous state. In contrast, the laser-cutting setting considered here is an open system. The temperature evolution depends not only on the previous temperature field but also on time-dependent forcing terms that encode the tool motion and power input.

Paper setting. In the paper setting [2], the neural operator learns a state transition of the form $T_{t-1}(\mathbf{x}) \mapsto T_t(\mathbf{x})$, optionally conditioned on low-dimensional parameters (e.g., global coef-

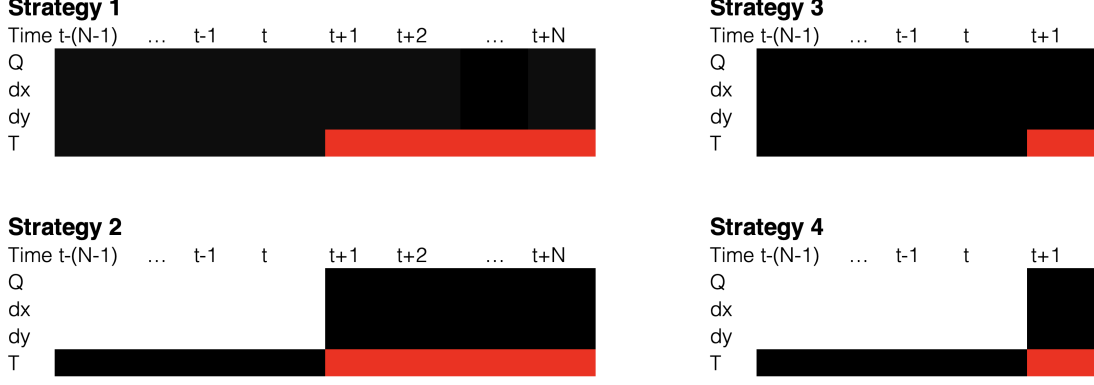


Figure 6: Black cells indicate inputs used as forcing terms (e.g., Q, dx, dy) and previous temperature fields T , red cells denote the target quantity that the model is trained to predict, and white cells mark inputs that are not provided for that configuration. The length of the returned sequence increases with N since the dataloader supplies correspondingly longer temporal windows.

ficients), which are typically incorporated via an embedding.

Modification Open-System Conditioning Unlike the closed-system setups typically used in [2], the laser-cutting dynamics depend on external forcing $f(\mathbf{x}, t)$. We therefore condition the neural operator on these forcing fields by concatenating them with the temperature input channels, so $f(\mathbf{x}, t)$ is available during both training and inference.

Modification Training Stabilization During training only, we add Gaussian noise to the input temperature to improve robustness,

$$T'_{t-1} = T_{t-1} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_T^2), \quad (3)$$

with σ_T given by `T_noise_sigma`. All other components (scheduler, objective, inference procedure) remain unchanged.

3.3.1 v-prediction

V-prediction [8] is employed to improve numerical stability and predictive accuracy while reducing the number of required refinement steps. In this formulation, the model predicts a linear combination of the clean signal and the injected noise rather than the noise directly. The predicted quantity at refinement step k is given by

$$v^k = \sqrt{1 - \sigma_k^2} \epsilon - \sigma_k u(t), \quad (4)$$

where $\epsilon \sim \mathcal{N}(0, 1)$ denotes Gaussian noise and $u(t)$ represents the target residual.

The noise level σ_k follows a power-law schedule defined as

$$\sigma_k = \sigma_{\min}^{\frac{k}{K_{\max}}}, \quad k = K_{\max}, K_{\max} - 1, \dots, 0, \quad (5)$$

with $\sigma_{\min} = \text{min_noise_std}$ denoting the minimum noise standard deviation and $K_{\max}$ the maximum number of refinement steps.

3.4 Metrics

3.4.1 Relative L_p Norms

To evaluate the accuracy of long-horizon rollouts and single-step predictions, we use scale-invariant relative errors based on the discrete L_p norm. For a ground-truth field y and a prediction $\hat{y}$, the relative L_p error is defined as

$$\text{rel } L_p(y, \hat{y}) = \frac{\|y - \hat{y}\|_p}{\|y\|_p}, \quad \|z\|_p = \left(\sum_i |z_i|^p \right)^{1/p},$$

where the sum runs over all grid points and we report results for $p \in \{1, 2\}$.

The choice of p provides complementary sensitivity. With $p = 1$, the metric summarizes the average absolute deviation over the field, whereas $p = 2$ weights larger local errors more strongly and is therefore more sensitive to localized instabilities [6].

3.4.2 Mean Squared Error

The mean squared error (MSE) is a regression loss that quantifies the average squared deviation between ground-truth targets and model predictions. For a set of N samples with targets y_i and predictions $\hat{y}_i$, it is defined as

$$\text{MSE}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N \|y_i - \hat{y}_i\|_2^2. \quad (6)$$

We employ MSE as the training loss because it is smooth and well-behaved for gradient-based optimization, provides stable training dynamics, and is a standard choice in operator-learning

literature, including the PDE-Refiner setup of Lippe et al. [2] [11, pp. 337–339].

3.4.3 Coefficient of Variation

The coefficient of variation (CV) quantifies relative dispersion by expressing the sample’s standard deviation as a fraction of its mean.

$$\text{CV} = \frac{\sigma}{\mu} \quad (7)$$

It is well suited for comparing the relative dispersion of quantities with different magnitudes or units, as it provides a normalized measure of variability. However, the CV is sensitive to strong trends and becomes ill-defined when the mean approaches zero. As a result, it may overestimate instability in cases where the curve exhibits a pronounced monotonic decrease despite smooth behavior [11, pp. 95–96].

3.4.4 Logarithmic-Transformed Mean Absolute Deviation

The mean absolute deviation is defined as the mean absolute first-order difference of a discrete time series. Applying the metric to log-transformed values yields a scale-invariant measure that emphasizes relative changes between successive samples.

$$\text{MAD}_{\log_{10}}(x) = \frac{1}{T} \sum_{t=1}^T |\log_{10}(x_{t+1}) - \log_{10}(x_t)|. \quad (8)$$

Compared to the coefficient of variation, the logarithmic-transformed mean absolute deviation (log-MAD) is less sensitive to strong non-stationary trends because it evaluates deviations in log space and therefore measures errors in multiplicative, dimensionless terms. This is advantageous when variability scales with the magnitude of the quantity, since relative changes are more informative than absolute ones and large values do not dominate the metric. In the context of training curves, a smooth but steep decrease can inflate the coefficient of variation even when short-term fluctuations are small, whereas log-MAD emphasizes local relative fluctuations around the trend and thus provides a more faithful measure of short-term training stability [11, pp. 336–337][12].

4 Experiments

This chapter introduces a baseline-based on the Fourier Neural Operator (FNO) and the Convolutional Neural Operator (CNO) and compares it with two diffusion-based refiner (DBR) approaches. The objective is to evaluate whether the DBR can outperform the baseline.

All experiments were implemented in Python using PyTorch and PyTorch Lightning. Training and evaluation were conducted on the ETH Zurich Euler cluster using NVIDIA RTX 4090 GPUs and 16-core CPUs, which provided the computational resources required for the hyperparameter studies and long-horizon rollout evaluations.

4.1 Baseline

The motivation for using a baseline is to have a benchmark of reliable and proven models which can be utilized to compare the results of the DBR implementation. The baseline gives insight on the training dynamic of Neural Operators in this case FNO and CNO. The parameter size of the models, training speed, training stability, and, most importantly, a comparable benchmark of rollout stability, rollout error, and error accumulation over time are evaluated. This is in line with the literature, where baseline models are used to evaluate the performance of a model [2], [3], [6].

4.1.1 Baseline - Experimental Setup

For the baseline experiments, a broad hyperparameter search was conducted to identify suitable combinations of dataloader strategies and sequence lengths N , with the aim of capturing general performance trends. The FNO and CNO models were initially trained for 500 epochs and evaluated using autoregressive rollouts of 150 time-steps. Based on these results, the best-performing hyperparameter configurations were selected and subsequently trained for 2000 epochs. The final evaluation was performed using extended rollouts of 165 time steps.

4.1.2 Baseline - Models

For both models, training is performed using the AdamW optimizer together with a CosineAnnealingLR learning rate scheduler, with identical hyperparameter configurations. An overview

of the selected hyperparameters is provided in Tables B.1 and B.2.

FNO is implemented as a three-layer architecture following the reference implementation extended to two-dimensional problems, as provided in [13]. The model is defined on a two-dimensional spatial domain and truncates the Fourier representation to the first 24 modes in each dimension. To mitigate boundary effects, reflective padding with a width of 8 is employed. The resulting network contains approximately 886 thousand trainable parameters.

CNO is implemented based on the reference implementation provided in [14]. The architecture consists of five layers, including four resolution levels in the encoder-decoder hierarchy and a central neck module. A channel multiplier of 16 is employed, resulting in approximately 8 million trainable parameters.

4.1.3 Baseline - Results

The broad hyperparameter search indicates that FNO is better suited for the laser cutting problem than CNO. A quantitative comparison of both models is provided in Table A.4.

For FNO, the average relative L_1 error is $\mu = 6.45\%$ with a standard deviation of $\sigma = 1.24\%$, while the average relative L_2 error is $\mu = 11.57\%$ with a standard deviation of $\sigma = 1.98\%$. This corresponds to a coefficients of variation (CV) of $CV = 0.19$ for the L_1 metric and $CV = 0.17$ for the L_2 metric, as defined in Sections 3.4.1 and 3.4.3.

In contrast, CNO achieves an average relative L_1 error of $\mu = 10.01\%$ with a standard deviation of $\sigma = 3.56\%$, and an average relative L_2 error of $\mu = 14.25\%$ with a standard deviation of $\sigma = 3.91\%$. The resulting coefficients of variation are $CV = 0.36$ for L_1 and $CV = 0.27$ for L_2 .

Overall, FNO not only achieves lower average error values but also exhibits significantly reduced variability across different experimental setups. Compared to FNO, CNO shows an 89% higher coefficient of variation for the L_1 metric and a 59% higher coefficient of variation for the L_2 metric. This indicates that FNO provides more stable and predictable performance for the considered laser-cutting problem.

Based on these findings, both FNO and CNO were selected for further experiments using Strategy 1 with sequence lengths $N = 2$ and $N = 3$. The models were subsequently trained for an extended duration of 2000 epochs and evaluated using rollouts of length 165 time steps. The corresponding results are summarized in Table 3.

Model	Strategy	N	L_1	L_2
FNO	1	3	2.83%	6.45%
FNO	1	2	3.23%	6.79%
CNO	1	3	5.25%	8.16%
CNO	1	2	5.95%	9.77%

Table 3: Relative L_1 and L_2 errors for a 165 time-step rollout in physical space after 2000 training epochs.

The FNO predictions remain stable along straight trajectories. However, the relative errors increase when the laser changes its direction, as illustrated by the test trajectory in Fig. 5. One possible explanation for the improved stability along straight paths is that the moving reference frame gradually reduces the influence of past temperature fields. As a result, sufficiently long straight trajectories resemble a quasi-stationary problem with a heat source located near the center of the domain. In contrast, trajectory turns introduce stronger temporal variations, which render the problem more dynamic and lead to increased prediction errors.

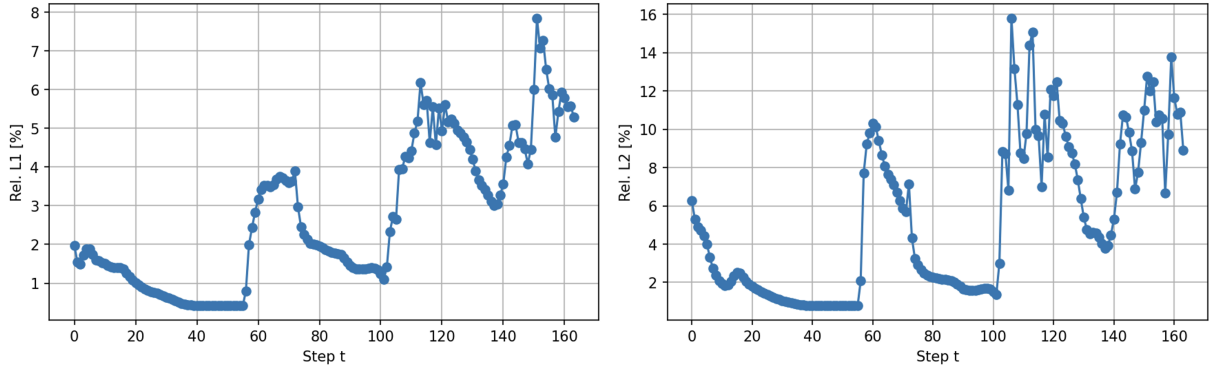


Figure 7: Relative L_1 and L_2 errors of the baseline FNO (Strategy 1, $N = 3$) during a 165 time-step rollout.

For the subsequent analysis, the configuration using FNO with Strategy 1 and $N = 3$ was chosen as the baseline, achieving a relative L_1 error of 2.83% and a relative L_2 error of 6.45%.

4.1.4 Baseline - Training Dynamics

The evolution of the training and validation losses shows that both FNO and CNO converge over the course of the training. After 2000 epochs, the validation loss of CNO converges to approximately 2×10^{-5} , while FNO reaches a lower value of about 1×10^{-5} .

Despite convergence in both cases, the training dynamics differ in terms of stability. Using the logarithmic-transformed mean absolute deviation (log-MAD) introduced in Section 3.4.4, CNO exhibits significantly higher variability in the validation loss, with a log-MAD of 11.8%,

compared to 2.2% for FNO. A similar trend is observed for the training loss, where CNO reaches a log-MAD of 5.7%, while FNO remains at 2.2%.

For FNO, the training and validation loss curves closely follow each other and converge after approximately 1000 epochs, indicating stable and consistent generalization behavior. In contrast, CNO shows a more pronounced discrepancy between training and validation losses. While the validation loss stabilizes after around 1400 epochs, the training loss continues to decrease until approximately 1650 epochs, suggesting less stable optimization dynamics. This change in behavior coincides with the reduction of the learning rate toward the end of training.

Overall, although both models converge successfully, FNO exhibits more stable and consistent training dynamics, whereas CNO shows increased short-term fluctuations. The corresponding loss curves are shown in Figures 8 and 9.

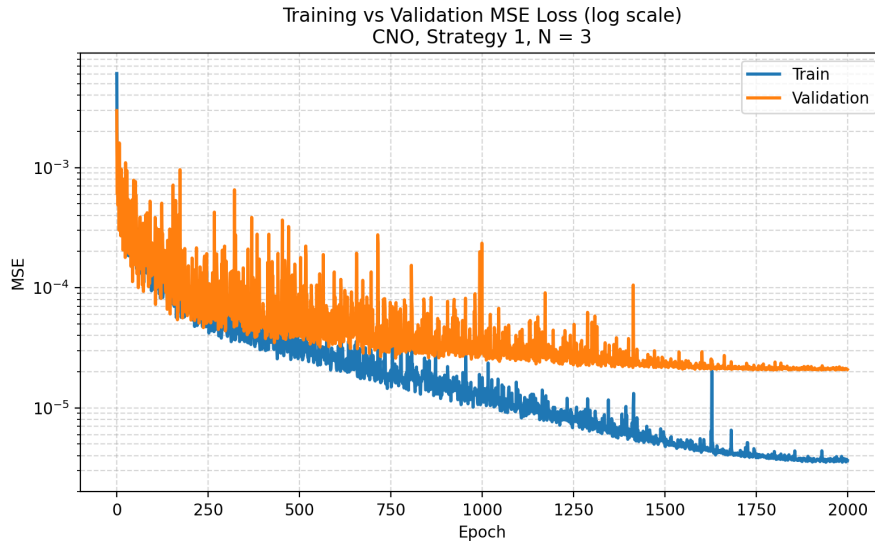


Figure 8: Training MSE, Validation MSE loss (log scale) for CNO Strategy 1, $N = 3$

4.2 Diffusion-Based Refiner - Modification Open-System Conditioning

This attempt investigates a configuration of the DBR without perturbations, based on the implementation of Lippe et al. [2], with minor adjustments discussed later. As shown in the following sections, this configuration exhibits highly sensitive and unstable behavior, with only a narrow range of hyperparameter settings leading to stable rollouts.

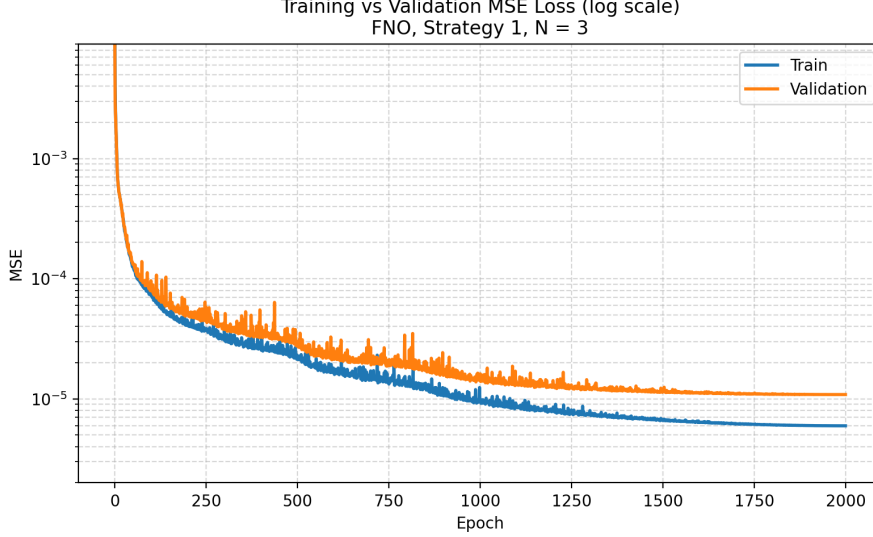


Figure 9: Training MSE, Validation MSE loss (log scale) for FNO Strategy 1, $N = 3$

4.2.1 Modification Open-System Conditioning - Experimental Setup

The experimental setup from the baseline is reused with minor modifications, see hyperparameters in Section B.3. Specifically, Strategy 3 with $N = 1$ is employed. While this configuration is equivalent to Strategy 1 with $N = 1$ in terms of the generated training samples, it conveniently facilitates loading and handling of the test data during evaluation.

In Modification Open-System Conditioning, the DBR is evaluated by varying the diffusion scheduler noise level `min_noise_std`, see Section 3.3.1 and the number of refinement steps K , see Section 2.2. A broad hyperparameter search is conducted over the ranges

$$\text{min_noise_std} \in \{1 \times 10^{-4}, 1 \times 10^{-5}, 1 \times 10^{-6}, 1 \times 10^{-7}\}, \quad K \in \{7, 10, 15\}.$$

All models are trained for 2000 epochs, and the final evaluation is performed using a rollout of length 165 time-steps, consistent with the baseline experiments.

4.2.2 Modification Open-System Conditioning - Model

The DBR employed in this work is based on the denoising diffusion probabilistic model (DDPM) framework proposed by Lippe et al. [2]. As the underlying neural operator, we use a Convolutional Neural Operator (CNO) with a time embedding, following [15]. The time embedding is essential because the predicted residual at each refinement step depends on the refinement index

K . In this sense, the refinement process can be interpreted analogously to a temporal evolution, where K acts as a discrete time-like variable.

Furthermore, we introduce modifications to the implementation of Lippe et al. [2]. In their implementation, the model receives the previous state together with a set of conditioning variables (e.g., the Reynolds number or a spatially uniform volumetric body force), which are incorporated via an embedding, whereas we pass the conditioning variables directly as additional input channels. We follow Lippe et al. [2] and use a CNO-based backbone instead of a FNO, since they report that FNO layers truncate high-frequency content and therefore perform suboptimally when predicting residual noise in diffusion-style denoising.

In our laser-cutting setting, the dynamics depend on the future tool trajectory and on the local laser input at specific spatial positions. Since the forcing terms (dx, dy, Q) in our dataset are provided on the same spatial grid as the temperature field T , we condition the model by concatenating these fields with the previous temperature T_{t-1} and feeding the resulting multi-channel tensor directly into the network.

Training The training implementation can be found in A.1. In each iteration, we first compute the residual between the previous temperature field T_{t-1} and the target field T_t ,

$$r = T_{t-1} - T_t. \quad (9)$$

A refinement step k is then sampled uniformly random in the interval $k \in [0, K]$, and Gaussian noise $\epsilon \sim \mathcal{N}(0, 1)$ with the same shape as r is generated. Using the DDPM scheduler with v -prediction in Section 3.3.1, we construct a noisy training target,

$$v_k = v_prediction(r, \epsilon, k). \quad (10)$$

Finally, the neural operator predicts the denoising residual for the refinement step r_k ,

$$r_k = NO(T_{t-1}, v_k, f, k), \quad (11)$$

where f denotes the forcing terms (dx, dy, Q) provided on the same spatial grid as T .

Inference The inference procedure is outlined in A.2. Starting from an initial noisy estimate y_0 , sampled from a standard normal distribution, the prediction is refined over K iterative denoising steps. At refinement step k with $k \in [0, K]$, the neural operator predicts a residual

update conditioned on the previous temperature field T_{t-1} , the current noisy state y_k , the forcing terms f , and the refinement index k ,

$$r_k = NO(T_{t-1}, y_k, f, k). \quad (12)$$

Subsequently, the DDPM scheduler performs one reverse-diffusion update, consistent with the training-time noise schedule, to obtain a denoised residual estimate $\hat{r}_k$. The refined prediction is then updated by adding this residual to the current state,

$$y_{k+1} = y_k + \hat{r}_k, \quad (13)$$

and the process is repeated until K refinement steps are completed.

4.2.3 Modification Open-System Conditioning - Results

The results indicate that this attempt exhibits unstable behavior, see Table 4. Only a narrow range of hyperparameter configurations yields stable rollouts, while most settings lead to divergence. Overall, the best-performing configuration does not surpass the baseline performance.

Stable rollouts are obtained for $\text{min_noise_std} \in \{1 \times 10^{-5}, 1 \times 10^{-6}\}$. In contrast, no consistent trend is observed with respect to the number of refinement steps K , increasing or decreasing K does not systematically improve the results.

Rel. L1 [%], Physical, 165 Rollout

min_noise_std	$K = 7$	$K = 10$	$K = 15$
1×10^{-4}	1.7×10^5	6.7×10^4	58.84
1×10^{-5}	13.69	29.65	64.32
1×10^{-6}	65.52	23.18	42.01
1×10^{-7}	1.3×10^{13}	64.59	173.58

Rel. L2 [%], Physical, 165 Rollout

min_noise_std	$K = 7$	$K = 10$	$K = 15$
1×10^{-4}	1.8×10^7	1.11×10^{10}	78.80
1×10^{-5}	25.17	61.97	79.02
1×10^{-6}	89.38	34.00	53.81
1×10^{-7}	6.2×10^{14}	339.94	267.41

Table 4: Hyperparameter for min_noise_std and the number of refinement steps K . Relative error in physical space, rollout length of 165.

The best-performing configuration in this experiment uses $\text{min_noise_std} = 1 \times 10^{-5}$ and

$K = 7$, yielding a relative L_1 error of 13.69% and a relative L_2 error of 25.17%. Compared to the baseline FNO with Strategy 1 and $N = 3$, this corresponds to an increase of approximately 484% in the relative L_1 error and 390% in the relative L_2 error.

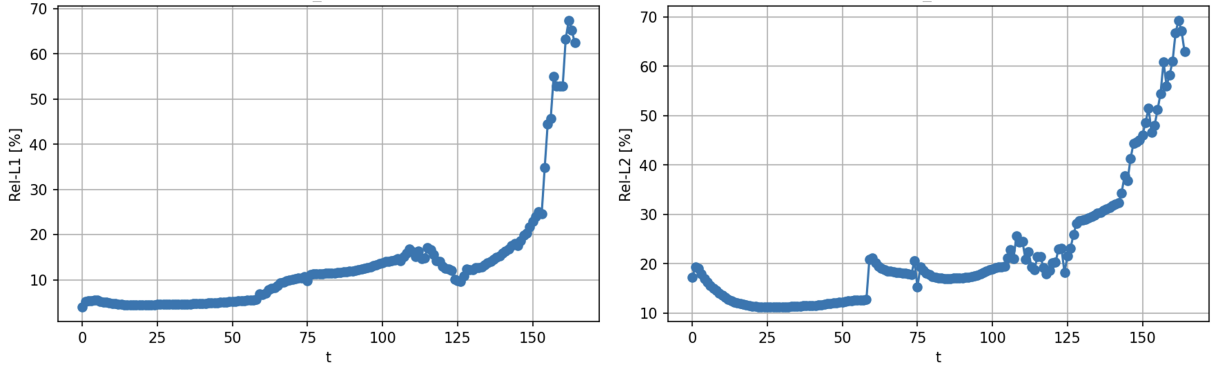


Figure 10: Relative L_1 and L_2 errors of the best Modification Open-System Conditioning ($\text{min_noise_std} = 1 \times 10^{-4}$, $K = 7$) during a 165 time-step rollout.

As shown in Fig. 10, the error begins to increase rapidly after approximately 140 time-steps. Compared to the baseline results in Fig. 7, where sharp changes in the trajectory lead to pronounced error spikes, such behavior is less dominant in this case. Instead, the error exhibits a smoother growth pattern that is consistent with approximately exponential error accumulation over time.

4.2.4 Modification Open-System Conditioning - Training Dynamics

The results indicate that the model begins to converge. However, the loss curves still exhibit a slight downward trend toward the end of training in Fig. 11. The reported training MSE is not directly comparable to the baseline training MSE, as we adopt the loss implementation of Lippe et al. [2]. In particular, the MSE is first averaged over the spatial dimensions and subsequently summed over time and channels, rather than averaged over all tensor elements as in PyTorch’s default MSE. The validation metric computation remains unchanged compared to the baseline.

For the validation curves, convergence is observed at approximately 5×10^{-5} . At this point, the model attains a log-MAD of 12.7%, which is close to the baseline CNO value of 11.8%. In contrast to Fig. 8, no noticeable damping of decay spikes is observed toward the end of training.

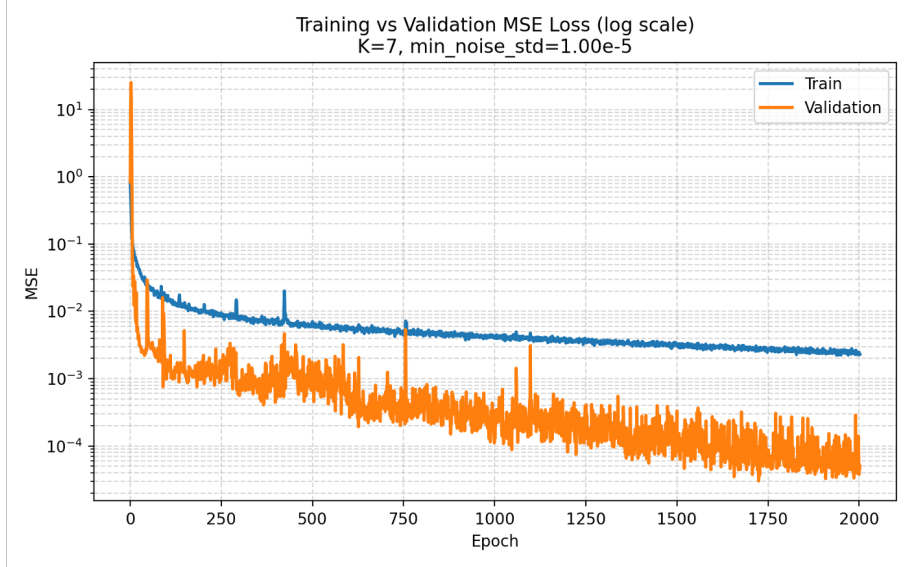


Figure 11: Training CustomMSE, Validation MSE loss (log scale) for $\text{min_noise_std} = 1 \times 10^{-5}$ and $K = 7$

4.3 Diffusion-Based Refiner - Modification Training Stabilization

Modification Training Stabilization augments the DBR with Gaussian input perturbations during training. We analyze the impact on long-horizon rollout stability and overall performance compared to Modification Open-System Conditioning and the baseline.

4.3.1 Modification Training Stabilization - Experimental Setup

The experimental setup from Modification Open-System Conditioning in 4.2.1 is reused with minor modifications, see the hyperparameters in B.3. The objective of this experiment is to assess whether adding a perturbation to the training input improves the long-horizon rollout performance and whether the resulting model can outperform the baseline benchmark.

Similar to Modification Open-System Conditioning, the diffusion scheduler is parameterized by min_noise_std , which controls the minimum noise magnitude in the schedule, and by K , which denotes the number of refinement steps. In addition, Modification Training Stabilization introduces the perturbation parameter T_noise_sigma , which controls the strength of the injected input noise during training.

In Modification Training Stabilization, we identified the following parameter ranges as promising

for further analysis:

$$\text{min_noise_std} \in \{1 \times 10^{-3}, 1 \times 10^{-4}, 1 \times 10^{-5}\}, \quad K \in \{7, 15, 25\},$$

$$\text{T_noise_sigma} \in \{1 \times 10^{-2}, 5 \times 10^{-3}, 1 \times 10^{-3}\}.$$

The model is trained for 2000 epochs, and the final evaluation is performed using a rollout of length 165 time-steps, consistent with the baseline experiments.

4.3.2 Modification Training Stabilization - Model

The DBR introduced in Modification Open-System Conditioning is largely retained. However, we add a perturbation term during training. The perturbation strength is controlled by the hyperparameter T_noise_sigma .

Training with perturbation We inject Gaussian noise with standard deviation $\sigma_T = \text{T_noise_sigma}$, yielding

$$T'_{t-1} = T_{t-1} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_T^2). \quad (14)$$

We then compute the residual using the perturbed input temperature field:

$$r = T'_{t-1} - T_t. \quad (15)$$

Apart from this perturbation, the training procedure and the inference pipeline remain unchanged.

4.3.3 Modification Training Stabilization - Results

The results indicate improved stability and overall performance compared to Modification Open-System Conditioning in Section 4.2.3. However, the method still does not outperform the baseline. In addition, the parameter search suggests that a broader range of hyperparameter configurations leads to non-divergent behavior, indicating increased robustness of the approach.

The best-performing configuration in this experiment uses $\text{min_noise_std} = 1 \times 10^{-3}$, $\text{T_noise_sigma} = 1 \times 10^{-2}$ and $K = 7$, yielding a relative L_1 error of 8.10% and a relative

min_noise_std	L_1			L_2		
	1×10^{-2}	5×10^{-3}	1×10^{-3}	1×10^{-2}	5×10^{-3}	1×10^{-3}
$K = 7$						
1×10^{-3}	8.10%	10.31%	12.52%	13.01%	15.48%	17.54%
1×10^{-4}	13.32%	8.50%	8.75%	19.18%	13.97%	14.85%
1×10^{-5}	76.67%	8.32%	54.35%	78.18%	14.48%	59.60%
$K = 15$						
1×10^{-3}	—	22.26%	—	—	27.51%	—
1×10^{-4}	—	16.22%	—	—	20.27%	—
1×10^{-5}	—	26.30%	—	—	31.62%	—
$K = 25$						
1×10^{-3}	—	—	—	—	—	—
1×10^{-4}	—	28.26%	—	—	34.89%	—
1×10^{-5}	—	—	—	—	—	—

Table 5: Modification Training Stabilization: Relative L_1 and L_2 scores in physical space (%). Rows: min_noise_std, columns: T_noise_sigma. Blocks correspond to different K . Rollout length 165 time-steps.

L_2 error of 13.01%. Compared to the baseline FNO with Strategy 1 and $N = 3$, this corresponds to an increase of approximately 286% in relative L_1 error and 202% in relative L_2 error.

It is noticeable that increasing the number of refinement steps K leads to worse relative L_1 and L_2 errors, see Fig. 12. This behavior contrasts with the findings of Lippe et al. [2], who report a saturation trend in which additional refinement steps initially yield substantial improvements, followed by diminishing returns as K increases further.

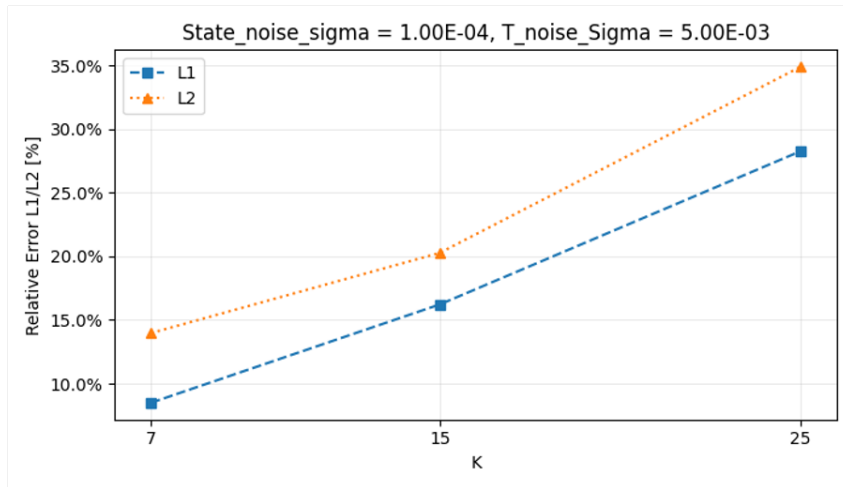


Figure 12: Relative L_1 and L_2 errors for min_noise_std = 1×10^{-4} , T_noise_sigma = 5×10^{-3}

Similar to the findings in Modification Open-System Conditioning, see Fig. 10, the error propagation in Modification Training Stabilization follows a comparable qualitative pattern, see Fig. 13. In particular, the rollout error begins to increase rapidly after approximately 140 time-steps, indicating the onset of divergence. However, the overall magnitude of the error remains lower than in Modification Open-System Conditioning, which is also reflected by the reduced average relative error metrics.

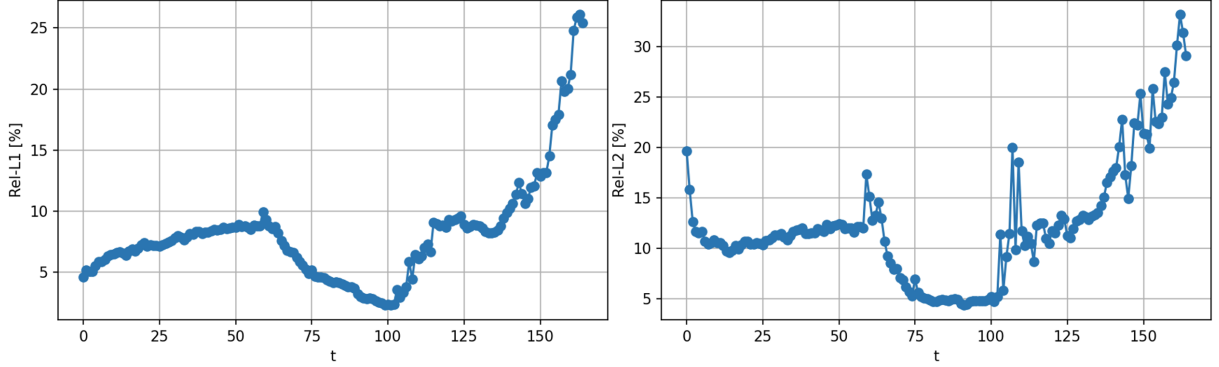


Figure 13: Relative L_1 and L_2 errors of the best Modification Training Stabilization for $\text{min_noise_std} = 1 \times 10^{-3}$, $T_{\text{noise_sigma}} = 1 \times 10^{-2}$, $K = 7$ during a 165 time-step rollout.

4.3.4 Modification Training Stabilization - Training Dynamics

For the validation curve, convergence occurs at a lower level of approximately 1.5×10^{-5} , compared to 5×10^{-5} in Section 4.2.4. Moreover, the validation dynamics are more stable, achieving a log-MAD of 11.3% versus 12.7% in Modification Open-System Conditioning.

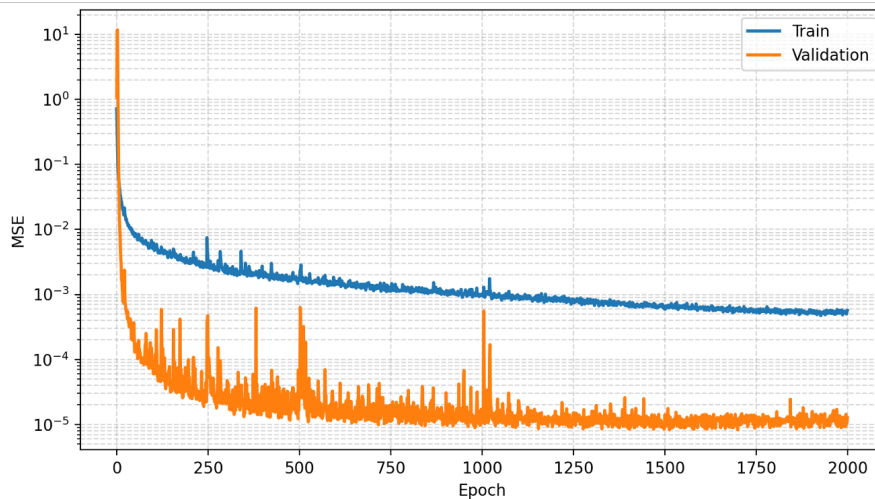


Figure 14: Training CustomMSE, Validation MSE loss (log scale) for $\text{min_noise_std} = 1 \times 10^{-3}$, $T_{\text{noise_sigma}} = 1 \times 10^{-2}$ and $K = 7$

4.4 Training and Inference Speeds

The baseline FNO and CNO experiments were conducted without mixed-precision arithmetic, which could further improve computational efficiency. Table 6 summarizes the measured training time per epoch and inference latency per predicted frame for all considered models.

As expected, the FNO is the most computationally efficient architecture, exhibiting both the fastest training time (5.5 s/epoch) and the lowest inference latency (0.9 ms). The CNO is substantially more expensive, which is consistent with its larger parameter count and higher computational complexity, resulting in 19.6 s/epoch for training and 5.0 ms per inference frame. The DBR lies between FNO and CNO in terms of training speed (9.8 s/epoch), but is significantly slower during inference (110 ms per frame). This increased inference cost is primarily attributed to the iterative refinement procedure, where multiple refinement steps are performed to produce a single prediction.

Model	Training speed	Inference speed per frame	Trainable Parameter
FNO	5.5 s/epoch	0.9 ms	886×10^3
CNO	19.6 s/epoch	5.0 ms	8.0×10^6
DBR	9.8 s/epoch	110 ms	41.9×10^6

Table 6: Training and inference speed comparison for FNO, CNO, and DBR. The DBR results are reported for $K = 7$ refinement steps. All timing measurements were obtained on a single NVIDIA RTX 4090 with 24 GB VRAM. The host system features a 16-core CPU with 2048 MB memory per core.

4.5 Summary of Results

Overall, the baseline neural operators provide the most reliable performance for the considered laser-cutting setting. In particular, the best baseline FNO configuration (Strategy 1, $N = 3$) achieves the lowest errors and the most stable long-horizon rollouts.

Modification Open-System Conditioning (DBR without input perturbation) shows weak generalization and highly unstable rollouts: most hyperparameter settings diverge, and even the best configuration remains substantially worse than the baseline. Modification Training Stabilization (DBR with input perturbation) improves rollout stability by injecting Gaussian noise into the input temperature during training, which yields consistently better results than Modification Open-System Conditioning and reduces divergence. However, it still does not outperform the baseline FNO. Additionally, increasing the number of refinement steps K tends to degrade performance in Modification Training Stabilization, contrary to the saturation behavior reported

by Lippe et al. [2].

Model	Relative L_1 [%]	Relative L_2 [%]
Best Baseline FNO (S1, $N = 3$)	2.83	6.45
Best Baseline CNO (S1, $N = 3$)	5.25	8.16
Best Modification Open-System Conditioning DBR	13.69	25.17
Best Modification Training Stabilization DBR	8.10	13.01

Table 7: Best results for baseline FNO and CNO and DBR variants (165 time-step rollout, physical space).

5 Conclusion

This thesis investigates whether diffusion-based refinement, as proposed in the PDE-Refiner approach by Lippe et al. [2], can improve long-horizon predictions of temperature fields in a laser-cutting setting. We evaluate two neural operator baselines (FNO and CNO) and compare them against two variants of the PDE-Refiner approach: one trained without input perturbations and one trained with perturbations.

5.1 Key Findings

The baseline FNO is the most accurate and stable. Across a broad hyperparameter range, FNO consistently achieved lower rollout errors and lower variability than CNO and the diffusion-based refinement approaches. For the final baseline configuration (Strategy 1, $N = 3$, 2000 epochs), FNO reached a relative L_1 error of 2.83% and a relative L_2 error of 6.45% over a 165 time-step rollout. In addition, the validation errors were markedly more stable for FNO, exhibiting substantially smaller short-term fluctuations as measured by the log-MAD.

Diffusion-based refinement is substantially more complex to train and less robust in our setting. In our experiments, diffusion-based refinement did not reproduce the improvements reported by Lippe et al. [2]. Several factors may explain this discrepancy.

First, the available training data in our setting is orders of magnitude smaller. We train on 1,934 samples, whereas Lippe et al. use 2,048 trajectories of the Kuramoto–Sivashinsky 1D equation with rollout length $140\Delta t$, yielding 286,720 samples in total. This difference is particularly relevant for diffusion-style models, which typically require large datasets to learn stable denoising dynamics.

Second, the underlying prediction task differs structurally. Our laser-cutting setup is an open system in which the laser trajectory influences the evolution of the temperature field. In contrast, the benchmark PDEs considered by Lippe et al. [2] are effectively closed systems without local external actuation. To better reflect this setting, dx , dy , and Q should be treated as conditioning variables that are embedded and forwarded through the network as additional inputs. In this way, the operator can still take the temperature field as its primary forwarded state, while the laser motion and power enter via embeddings used consistently during both training and inference.

Third, the DBR solves a substantially harder learning problem. While the baselines learn a deterministic evolution of a relatively smooth parabolic PDE, the DBR must approximate a reverse-time diffusion process via multiple denoising steps, requiring accurate residual/noise prediction across many noise levels and stable iterative refinement. Consequently, DBR tend to be more sensitive to architectural choices and optimization settings, and typically require more careful tuning (and often more data) to train reliably.

Diffusion-based refinement benefits from perturbations during training. We show that augmenting the training inputs with perturbations substantially improves training stability, reduces sensitivity to hyperparameters, and yields better long-horizon rollout performance. In our experiments, perturbation reduces the relative errors from 13.69% to 8.10% in L_1 and from 25.17% to 13.01% in L_2 .

The refiner is significantly more expensive at inference time. Even when training time per epoch was moderate, inference was dominated by iterative refinement. FNO achieved the fastest inference (0.9 ms per frame), whereas the PDE-Refiner required approximately 110 ms per frame, reported for $K = 7$, making the refinement approach substantially less attractive for fast surrogate deployment in this setting.

More refinement steps did not improve performance. Contrary to the saturation trend reported by Lippe et al. [2], increasing the number of refinement steps K degraded rollout accuracy in our experiments.

5.2 Limitations

Several limitations constrain the interpretation of these results. First, the integration of diffusion-style FNO backbone was not pursued due to architectural complexity of implementing a time embedding, which prevented evaluating whether a DBR could benefit from the stronger FNO operator as its denoiser. Second, the dataset is small, and the laser-cutting dynamics include strong non-stationarities from trajectory changes. Both aspects may amplify sensitivity to noise schedules and refinement hyperparameters. Finally, the refiner was trained and evaluated in a largely fixed pipeline. Alternative training schemes (e.g., more autoregressive training, different conditioning mechanisms, or larger-scale datasets) may change the outcome.

5.3 Future Work

The results suggest three directions for future work. First, implementing and evaluating diffusion-based refinement with an FNO-based backbone, including an appropriate time embedding, which would test whether the refiner’s limitations are primarily architectural rather than conceptual. Second, improved training pipelines may be required for refinement to succeed in this setting, including larger and more diverse datasets, explicit autoregressive training objectives, and more systematic hyperparameter optimization of the diffusion schedule. Third, rather than adapting a neural operator backbone, future work could explore diffusion architectures designed specifically for DDPM-style denoising in spatio-temporal fields, potentially improving stability.

5.4 Overall Conclusion

In summary, this thesis benchmarked FNO and CNO baselines and implemented a DBR for laser-cutting temperature prediction. The experiments show that FNO provides the best combination of rollout accuracy, stability, and computational efficiency for this problem. The implemented DBR did not improve long-horizon rollouts in this setting, was highly sensitive to hyperparameters, and incurred a substantial inference-time cost. Adding perturbations to the training improved robustness, but the refined rollouts still lagged behind the FNO baseline.

6 Acknowledgment

I would like to express my sincere gratitude to Prof. Dr. Laura De Lorenzis for her support in finding this thesis project and for connecting me with my supervisor. I am especially grateful to my supervisor, Lukas, for his continuous guidance and the generous time he invested in helping me, often for several hours at a time. I also thank for granting me access to the Euler cluster, which significantly accelerated my research and enabled efficient execution of the computational experiments in this thesis.

A Appendix

A.1 Modification Open-System Conditioning Pseudo Code

Listing 1: Training step of the DBR (without pertubation)

```
1 def train_step(x, y, cond):
2     # Residual between target and previous state (sequence length N)
3     residual = y - x
4
5     # Sample diffusion timestep k uniformly from {0, ..., K_max} (per
6     # batch element)
7     k = randint(low=0, high=K_max + 1, size=batch_size(x))
8
9     # Gaussian noise with same shape as the residual
10    eps = randn_like(residual)
11
12    # Forward diffusion: add noise according to the scheduler
13    y_noised = scheduler.add_noise(residual, eps, k)
14
15    # Build model input by concatenating along the channel dimension
16    x_in = concat([x, y_noised, cond], dim=channels)
17
18    # Predict v-parameterization (time embedding uses scaled timestep)
19    pred = neural_operator(x_in, time=k * time_multiplier)
20
21    # Compute v-prediction target from residual and noise
22    target = scheduler.get_velocity(residual, eps, k)
23
24    # Training loss
25    loss = train_criterion(pred, target)
26
27    return loss
```

A.2 Modification Training Stabilization Pseudo Code

Listing 2: Training step of the DBR (with pertubation)

```
1 def train_step(x, y, cond):
2     # Add perturbation to the input state
3     x_noisy = x + state_noise_sigma * randn_like(x)
4
5     # Residual between target and previous state (sequence length N)
6     residual = y - x_noisy
7
8     # Sample diffusion timestep k uniformly from {0, ..., K_max} (per
9         batch element)
10
11     k = randint(low=0, high=K_max + 1, size=batch_size(x))
12
13
14     # Gaussian noise with same shape as the residual
15     eps = randn_like(residual)
16
17     # Forward diffusion: add noise according to the scheduler
18     y_noised = scheduler.add_noise(residual, eps, k)
19
20     # Build model input by concatenating along the channel dimension
21     x_in = concat([x, y_noised, cond], dim=channels)
22
23     # Predict v-parameterization (time embedding uses scaled timestep)
24     pred = neural_operator(x_in, time=k * time_multiplier)
25
26     # Compute v-prediction target from residual and noise
27     target = scheduler.get_velocity(residual, eps, k)
28
29     # Training loss
30     loss = train_criterion(pred, target)
31
32     return loss
```

A.3 Inference Pseudo Code

Listing 3: Inference procedure of the DBR (iterative denoising)

```
1 def inference(x, cond):
2     # Initialize noisy estimate for the future window
3     y_noised = randn(shape=(batch_size(x), T_future, spatial_shape(x)),
4                       dtype=dtype(x), device=device(x))
5
6     # Set reverse-diffusion timesteps (K_max refinement steps)
7     scheduler.set_timesteps(K_max, device=device(x))
8
9     for k in scheduler.timesteps:
10        # Time embedding (scaled timestep)
11        time = full(shape=(batch_size(x),), fill_value=k,
12                     dtype=dtype(x), device=device(x))
13
14        # Concatenate previous state, current noisy estimate, and
15        # conditioning
16        x_in = concat([x, y_noised, cond], dim=channels)
17
18        # Predict residual update at step k
19        pred = neural_operator(x_in, time=time * time_multiplier)
20
21        # Reverse-diffusion update
22        y_noised = scheduler.step(pred, k, y_noised).prev_sample
23
24    # Add y_noised to denoise input x
25    return x + y_noised
```

A.4 Baseline Results

Strategy	N	Model	L1	L2	Model	L1	L2
Strategy 1	4	CNO	15.35%	19.08%	FNO	7.20%	11.06%
Strategy 1	3	CNO	6.70%	9.31%	FNO	5.38%	9.31%
Strategy 1	2	CNO	5.18%	8.34%	FNO	5.39%	9.99%
Strategy 2	4	CNO	7.80%	10.63%	FNO	6.28%	10.70%
Strategy 2	3	CNO	12.00%	15.59%	FNO	5.75%	9.42%
Strategy 2	2	CNO	13.65%	20.65%	FNO	4.84%	8.93%
Strategy 3	4	CNO	10.31%	15.47%	FNO	6.17%	11.66%
Strategy 3	3	CNO	5.77%	10.47%	FNO	6.87%	14.14%
Strategy 3	2	CNO	8.09%	13.18%	FNO	6.13%	12.16%
Strategy 3	1	CNO	7.79%	13.39%	FNO	5.31%	10.27%
Strategy 4	4	CNO	12.84%	17.30%	FNO	8.37%	14.11%
Strategy 4	3	CNO	11.96%	16.65%	FNO	7.19%	13.16%
Strategy 4	2	CNO	15.73%	18.36%	FNO	6.12%	11.83%
Strategy 4	1	CNO	7.00%	10.75%	FNO	9.29%	15.21%

Table 8: Relative L_1 and L_2 errors for a 150 time-step rollout in physical space after 500 training epochs. Strategy refers to the dataloader configurations described in Section 3.2.2.

B Hyperparameters

B.1 FNO Baseline Hyperparameter

Table 9: Hyperparameters used for the FNO baseline training

Parameter	Value
Epochs	500
Seed	0
Validation Split	0.2
Batch Size	16
Width	16
Modes (first dimension)	24
Modes (second dimension)	24
Padding	8
Fourier Layers	3
Model Parameter Size	885K
Optimizer	AdamW
Optimizer LR	0.001
Optimizer Weight Decay	0.01
Scheduler	CosineAnnealingLR

B.2 CNO Baseline Hyperparameter

Table 10: Hyperparameters used for the CNO baseline training

Parameter	Value
Epochs	500
Seed	0
Validation Split	0.2
Batch Size	16
Size	44
Channel Multiplier	16
N Layers	5
N Res	4
N Res Neck	4
Model Parameter Size	8.0M
Optimizer	AdamW
Optimizer LR	0.001
Optimizer Weight Decay	0.01
Scheduler	CosineAnnealingLR

B.3 DBR Hyperparameter

Table 11: Hyperparameters used for Attempt 1 and Attempt 2

Parameter	Value
Epochs	2000
Seed	0
Validation Split	0.2
Batch Size	32
Size	44
Channel Multiplier	32
N Layers	5
N Res	2
N Res Neck	5
Model Parameter Size	41.9M
Optimizer	AdamW
Optimizer LR	0.001
Optimizer Weight Decay	0.01
Scheduler	CosineAnnealingLR
Scheduler Eta Min	1×10^{-5}
Exponential Moving Average	0.995

All remaining model hyperparameters follow the default settings of the reference implementation provided in [16].

Bibliography

- [1] N. Kovachki, Z. Li, B. L. and Kamyar Azizzadenesheli, K. Bhattacharya, and A. S. and Anima Anandkumar, “Neural operator: Learning maps between function spaces with applications to pdes,” *arXiv preprint arXiv:2108.08481*, 2022. [Online]. Available: <https://arxiv.org/abs/2108.08481>.
- [2] P. Lippe, B. S. Veeling, P. Perdikaris, R. E. Turner, and J. Brandstetter, “Pde-refiner: Achieving accurate long rollouts with neural pde solvers,” *arXiv preprint arXiv:2308.05732*, 2023. [Online]. Available: <https://arxiv.org/abs/2308.05732>.
- [3] Z. Li, N. Kovachki, K. Azizzadenesheli, *et al.*, “Fourier neural operator for parametric partial differential equations,” *arXiv preprint arXiv:2010.08895*, 2021. [Online]. Available: <https://arxiv.org/abs/2010.08895>.
- [4] T. Chen and H. Chen, “Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its applications to dynamic systems,” *Neural Networks, IEEE Transactions on*, pp. 911–917, Aug. 1995. DOI: [10.1109/72.392253](https://doi.org/10.1109/72.392253).
- [5] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, “Learning nonlinear operators via deepnet based on the universal approximation theorem of operators,” *Nature Machine Intelligence*, vol. 3, no. 3, pp. 218–229, Mar. 2021, ISSN: 2522-5839. DOI: [10.1038/s42256-021-00302-5](https://doi.org/10.1038/s42256-021-00302-5). [Online]. Available: <http://dx.doi.org/10.1038/s42256-021-00302-5>.
- [6] B. Raonić, R. Molinaro, T. D. Ryck, *et al.*, “Convolutional neural operators for robust and accurate learning of pdes,” *arXiv preprint arXiv:2302.01178*, 2023. [Online]. Available: <https://arxiv.org/abs/2302.01178>.
- [7] F. Bartolucci, E. de Bezenac, B. Raonic, R. Molinaro, S. Mishra, and R. Alaifari, “Representation equivalent neural operators: A framework for alias-free operator learning,” 2023. [Online]. Available: <https://openreview.net/forum?id=7LSEkvEGCM>.
- [8] T. Salimans and J. Ho, “Progressive distillation for fast sampling of diffusion models,” 2022. arXiv: [2202.00512](https://arxiv.org/abs/2202.00512) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2202.00512>.
- [9] D. Poulikakos, *Thermodynamik und Wärmetransport*. 2023, Lecture notes thermodynamics 3, in German.

- [10] C. A. Moreira, M. A. Caicedo, M. Cervera, M. Chiumenti, and J. Baiges, “A multi-criteria h-adaptive finite-element framework for industrial part-scale thermal analysis in additive manufacturing processes,” *Engineering with Computers*, pp. 4791–4813, 2022. DOI: [10.1007/s00366-022-01655-0](https://doi.org/10.1007/s00366-022-01655-0). [Online]. Available: <https://doi.org/10.1007/s00366-022-01655-0>.
- [11] Y. Dodge, Ed., *The Concise Encyclopedia of Statistics*. Springer, 2008, ISBN: 978-0-387-32833-1. DOI: [10.1007/978-0-387-32833-1](https://doi.org/10.1007/978-0-387-32833-1).
- [12] B. N. Seegers, R. P. Stumpf, B. A. Schaeffer, K. A. Loftin, and P. J. Werdell, “Performance metrics for the assessment of satellite data products: An ocean color case study,” *Optics Express*, vol. 26, no. 6, pp. 7404–7422, 2018. DOI: [10.1364/OE.26.007404](https://doi.org/10.1364/OE.26.007404).
- [13] CAMLab ETH Zurich, *Tutorial 05 - Operator Learning - Fourier Neural Operator*, https://github.com/camlab-ethz/AI_Science_Engineering/blob/main/Tutorial%205%20-%20Operator%20Learning%20-%20Fourier%20Neural%20Operator.ipynb, GitHub repository, accessed Sep. 9, 2025, 2023.
- [14] CAMLab ETH Zurich, *Simplified Convolutional Neural Operator Repository*, https://github.com/camlab-ethz/ConvolutionalNeuralOperator/tree/main/CN02d_simplified, GitHub repository, accessed Oct. 06, 2025, 2023.
- [15] M. Herde, B. Raonić, T. Rohner, *et al.*, “Poseidon: Efficient foundation models for pdes,” 2024. arXiv: [2405.19101](https://arxiv.org/abs/2405.19101) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2405.19101>.
- [16] CAMLab ETH Zurich, *Temporal Convolutional Neural Operator Repository*, https://github.com/camlab-ethz/ConvolutionalNeuralOperator/tree/main/CN02d_temporal, GitHub repository, accessed Nov. 30, 2025, 2023.